

## Situation 3 : Gestion des versions avec Git et Github

 **Fiche rédigée par :** GADONNAUD Ewen

 **Formation :** BTS SIO 2ème année - Option SISR

 **Établissement :** Lycée Paul-Louis Courier, Tours

 **Date :** Septembre 2026



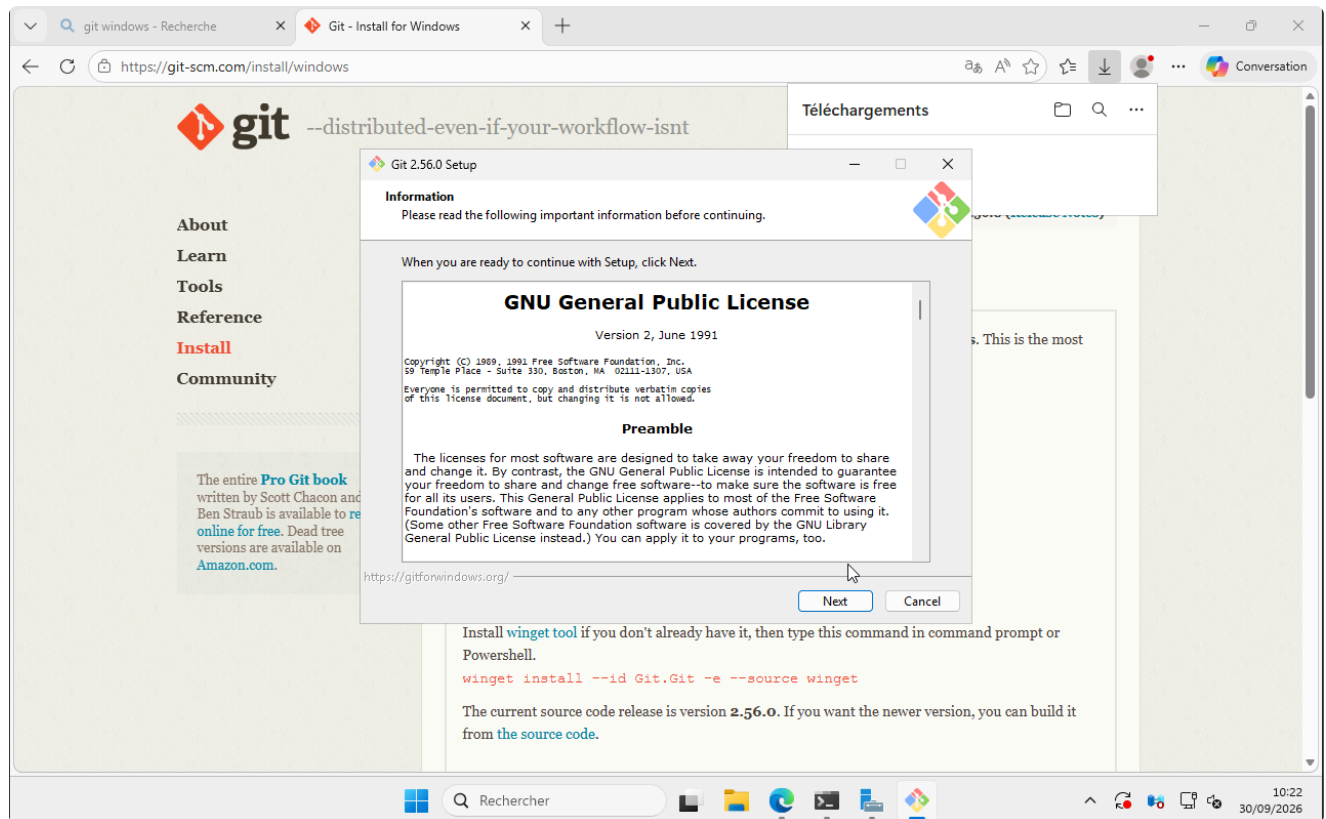
---

### Partie 1 - Installation et configuration de Git

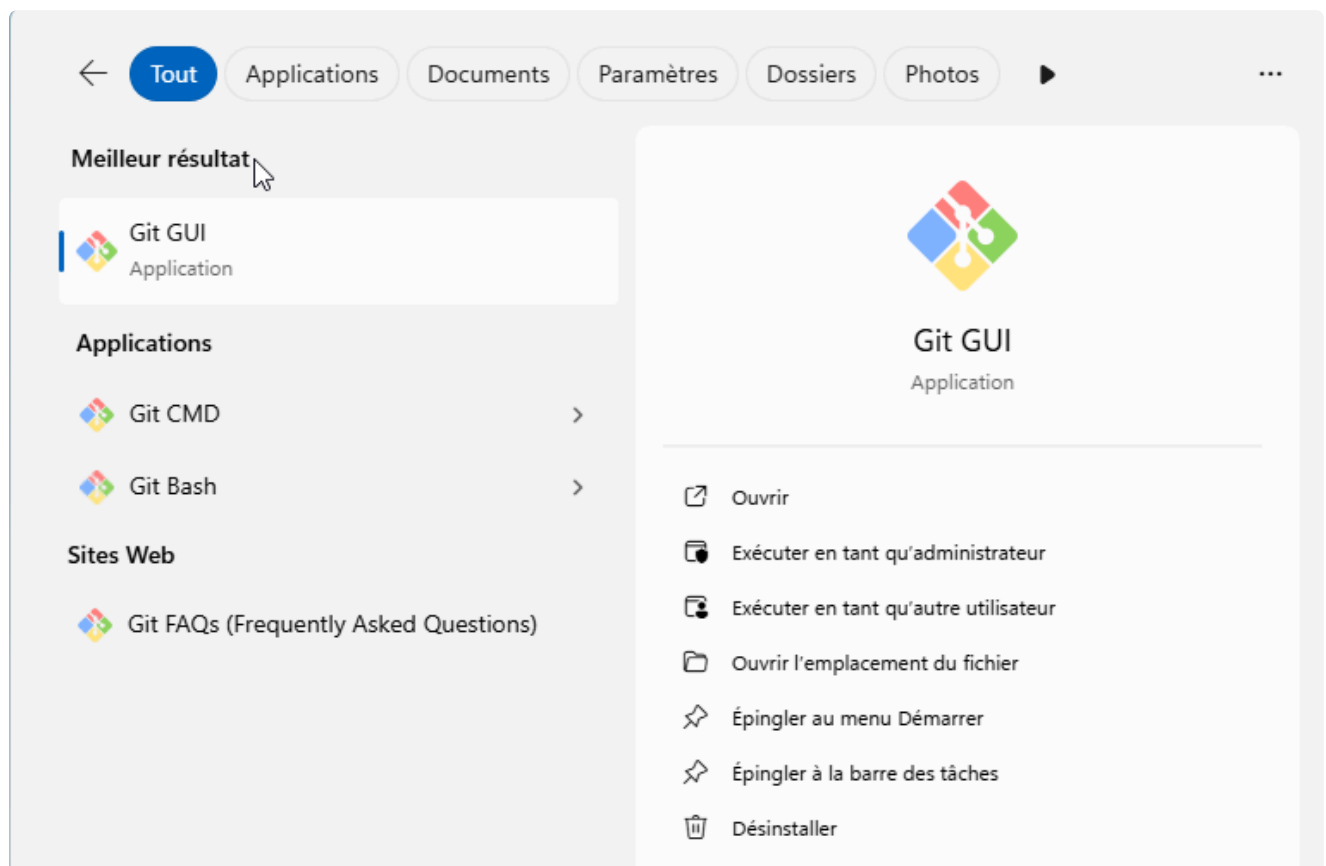
**1. Installer la version windows de Git sur votre poste ou serveur d'administration**

Ici, Git et toute la situation se déroulera sur le serveur WAC0 d'administration.

Nous installons git sur l'URL suivante : <https://git-scm.com/install/windows>



Maintenant, vérifions que Git Bash, Git GUI et Git CMD soient installés :



La version de Git actuellement installée est la version 2.56.0.windows.1

Git est utilisé afin de pouvoir versionner du code, des fichiers et autres types de documents. Le versionnage consiste à prendre une sorte d'instantané à un moment X de notre fichier versionné et pouvoir revenir en arrière vers d'autres versions de notre code quand nous le souhaitons. Il est aussi utilisé dans le travail collaboratif, notamment à l'aide des PRs (Pull Request) et des branches.

## 2. Vérifier puis configurer l'identité utilisée par Git :

Nous utilisons les commandes suivantes afin de configurer notre identité utilisée par Git :

```
$ git config --global user.name "Ewen Gadonnaud"
$ git config --global user.email "eweng.pro@gmail.com"
$ git config --global --list

user.name=Ewen Gadonnaud
user.email=eweng.pro@gmail.com
```

Git associe-t-il un nom et une adresse électronique aux commits afin de pouvoir assurer une traçabilité concrète et d'y permettre une identification quant à qui à fait quoi et quand lors de changements des versions d'un fichier ou autre dans un dépôt Git.

C'est pour quoi il est nécessaire de configurer son identité au préalable.

## Partie 2 - Création d'un premier dépôt Git

Après création du script "diagnostic-reseau.ps1" contenant ce bout de code :

```
Write-Host "Diagnostic réseau CUB"
hostname
Get-Date
Get-NetIPConfiguration
```

Nous initialisons depuis Git Bash, en se plaçant dans le répertoire ou se trouve notre bout de code, à l'aide des commandes suivantes :

```

ADM-SRV-WACO@ServeurWACO UCRT64 /c/CUB/Scripts
$ ls
diagnostic-reseau.ps1

ADM-SRV-WACO@ServeurWACO UCRT64 /c/CUB/Scripts
$ git init
Initialized empty Git repository in C:/CUB/Scripts/.git/

ADM-SRV-WACO@ServeurWACO UCRT64 /c/CUB/Scripts (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    diagnostic-reseau.ps1

nothing added to commit but untracked files present (use "git add" to track)

ADM-SRV-WACO@ServeurWACO UCRT64 /c/CUB/Scripts (master)
$ |

```

L'initialisation d'un dépôt Git constitue notre base de code ou fichiers qui sera versionné dans le futur, il contiendra tout nos commits, nos branches, et nos PRs.

La commande "git status" nous permet de savoir si des commits ont été effectués sur le dépôt, de savoir sur quelle branche de développement nous nous situons et des éventuels fichiers résiduels non ajoutés en commit et qui ne sont pas encore suivis par Git.

## Partie 3 - Création des premières versions du script

Nous ajoutons notre fichier au suivi de Git avec la commande suivante :

```
git add diagnostic-reseau.ps1
```

Notre commande "git status" nous renvoie la sortie suivante :

```

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   diagnostic-reseau.ps1

```

en nous expliquant que le fichier "diagnostic-reseau.ps1" est bien ajouté en phase de transition pré commit, mais qu'il reste à faire le commit.

Nous créons alors notre premier commit commenté avec la commande suivante :

```
git commit -m "Création du script de diagnostic réseau"
```

# Ici le paramètre -m correspond à l'ajout d'un commentaire pour notre commit qui sera visible par nos collaborateurs sur le dépôt.

Et avec la commande "git log" et "git log --oneline", nous vérifions l'historique de nos commits :

```

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git commit -m "Création du script de diagnostic réseau"
[master (root-commit) dbd3a2a] Création du script de diagnostic réseau
1 file changed, 4 insertions(+)
create mode 100644 diagnostic-reseau.ps1

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git log
commit dbd3a2a8ac06f2dee0cdf65d7188bf36586a2b55 (HEAD -> master)
Author: Ewen Gadonnaud <eweng.pro@gmail.com>
Date:   Wed Sep 30 10:51:14 2026 +0200

    Création du script de diagnostic réseau

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git log --oneline
dbd3a2a (HEAD -> master) Création du script de diagnostic réseau

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$

```

Un commit est une phase transitoire entre l'étape d'ajout d'un fichier qui sera versionné (git add) et le poussage vers le dépôt distant ou local Git de ce fichier versionné (git push). Le commit est utilisé pour informer du changement de version de notre fichier versionné, avec un commentaire.

Il est important d'utiliser un message de commit précis et explicite notamment en projet de groupe, afin d'être le plus informatif possible et de savoir d'un coup d'oeil les fonctionnalités principales ayant changées, les bugs fix etc...

## Partie 4 - Modification et suivi d'un fichier

Nous modifions le fichier "diagnostic-reseau.ps1" en ajoutant le code suivant :

```

Write-Host "Test de la pile TCP/IP"
ping 127.0.0.1

```

et nous vérifions les modifications avec les commandes "git status" et "git diff"

La commande "git diff" permet de comparer les changements entre un fichier local et un fichier qui est déjà en commit/push

```

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   diagnostic-reseau.ps1

no changes added to commit (use "git add" and/or "git commit -a")

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git diff
diff --git a/diagnostic-reseau.ps1 b/diagnostic-reseau.ps1
index 487d6de..41fefe7 100644
--- a/diagnostic-reseau.ps1
+++ b/diagnostic-reseau.ps1
@@ -1,4 +1,6 @@
 Write-Host "Diagnostic Réseau CUB"
 hostname
 Get-Date
-Get-NetIPConfiguration
\ No newline at end of file
+Get-NetIPConfiguration
+Write-Host "Test de la pile TCP/IP"
+ping 127.0.0.1
\ No newline at end of file

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ |

```

Nous enregistrons ensuite cette nouvelle version en utilisant les commandes précédemment utilisées pour faire un commit :

```

git add diagnostic-réseau.ps1
git commit -m "Ajout du test TCP IP local"
git log --oneline

```

```

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git add diagnostic-reseau.ps1

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git commit -m "Ajout du test TCP IP local"
[master 6c23afb] Ajout du test TCP IP local
1 file changed, 3 insertions(+), 1 deletion(-)

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git log --oneline
6c23afb (HEAD -> master) Ajout du test TCP IP local
dbd3a2a Création du script de diagnostic réseau

```

A présent, nous avons deux versions de notre projet déjà présents dans notre historique (6c23afb et dbd3a2a).

Nous réalisons une troisième modification en ajoutant les lignes suivantes :

```

Write-Host "Affichage des serveurs DNS"
Get-DnsClientServerAddress

```

Puis nous faisons le troisième commit :

```
git add diganostic-reseau.ps1
git commit -m "Ajout du diagnostic DNS"
git log --oneline
```

```
ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git log --oneline
87b97df (HEAD -> master) Ajout du diagnostic DNS
6c23afb Ajout du test TCP IP local
dbd3a2a Création du script de diagnostic réseau

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$
```

## Partie 5 - Comprendre le fonctionnement de Git

Explication des différentes commandes et de leurs influences sur le cycle de versionnement utilisé par Git :

- git status : vérifier notre position dans le dépôt (quelle branche), l'état des fichiers indexés ou non indexés.
- git diff : comparer les changements entre un fichier local et un fichier qui est déjà sur le dépôt git local
- git add : envoyer les fichiers vers la zone d'indexation, juste avant l'envoi vers le dépôt Git local grâce au git commit
- git commit : envoyer les fichiers vers le dépôt Git local avec un message précisant les modifications apportés à ce/ces dernier(s)
- git log : vérifier les différentes versions d'un projet existant et l'auteur des différentes modifications

## Partie 6 - Annulation d'une modification non validée

Nous ajoutons volontairement une ligne dans le fichier de script sans créer de commit afin de pouvoir essayer la commande "git restore" :

```
Write-Host "TEST INCORRECT"
```

Nous vérifions les modifications et choisissons de restaurer le fichier diagnostic-reseau.ps1 :

```
git diff
git restore diagnostic-reseau.ps1
```



```

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git diff
diff --git a/diagnostic-reseau.ps1 b/diagnostic-reseau.ps1
index 0f85274..b513771 100644
--- a/diagnostic-reseau.ps1
+++ b/diagnostic-reseau.ps1
@@ -5,4 +5,5 @@ Get-NetIPConfiguration
 Write-Host "Test de la pile TCP/IP"
 ping 127.0.0.1
 Write-Host "Affichage des serveurs DNS"
-Get-DnsClientServerAddress
\ No newline at end of file
+Get-DnsClientServerAddress
+Write-Host "TEST INCORRECT"
\ No newline at end of file

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git restore diagnostic-reseau.ps1

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git diff

```

La commande git restore permet donc de récupérer la dernière version d'un fichier présente dans le dépôt git local et d'écraser la version locale.

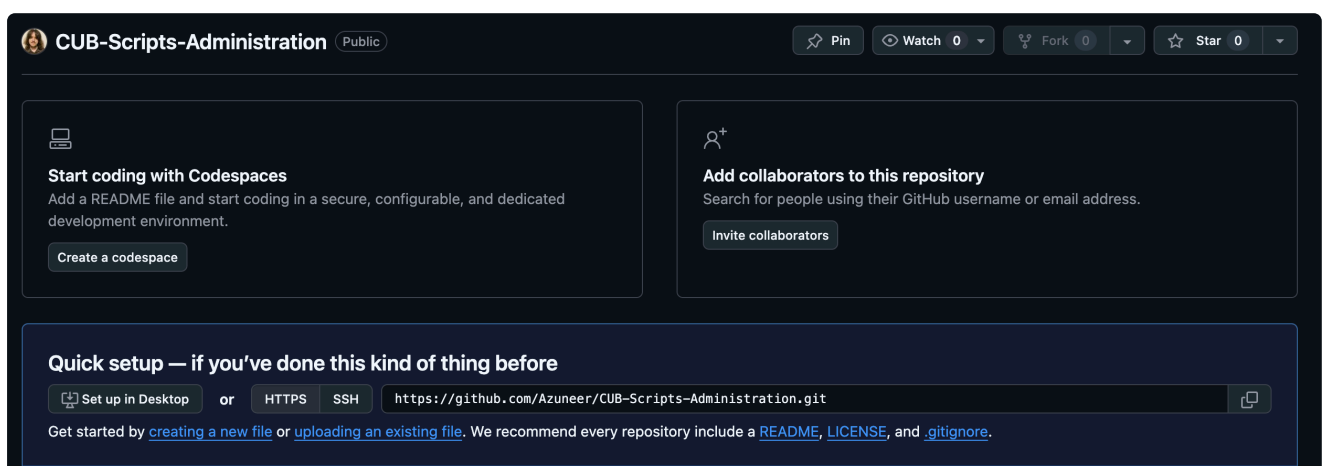
La modification que l'on avait faite n'a pas été enregistrée dans un commit car nous n'avons ni utilisés la commande "git add" ni "git commit".

## Partie 7 - Découverte de GitHub

Il est possible d'utiliser Git sans GitHub, GitHub permet seulement de mettre son dépôt en ligne sur les serveurs de GitHub, ce qui est utile car il peut agir comme un "hub" de sauvegardes de projet, alors qu'avec Git, tout est fait en local.

Git est un outil de versionning en local, tandis-ce que GitHub permet d'interconnecter tout les dépôts de plusieurs millions d'utilisateurs dans le monde entier.

Nous créons alors le dépôt GitHub "CUB-Scripts-Administration" en visibilité publique.



Un dépôt public est un dépôt visible et accessible à tous, il peut être cloné localement par n'importe qui. Un dépôt privé quant à lui n'est pas visible par tous, et n'est pas clonable par n'importe qui.



Un administrateur système doit être vigilant avant de publier un script sur un dépôt public car il peut donner des informations sensibles en lien avec l'infrastructure de l'entreprise dans laquelle il travaille, que ce soit des mots de passes ou adresses IP locales ou autres..

Sur un dépôt GitHub il ne faut publier :

- Clés APIs
- Mots de passes
- Adresses IPs d'équipements sensibles

## Partie 8 - Relier le dépôt local au dépôt GitHub

Avec la commande suivante, nous vérifions les dépôts distants déjà configurés :

```
git remote -v
```

Ici, aucun n'est configuré.

Nous associons donc notre dépôt local au dépôt GitHub créé précédemment :

```
git remote add origin https://github.com/Azuneer/CUB-Scripts-Administration.git
git remote -v
```

Le nom origin représente le nom de base du dépôt distant configuré.

```
ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git branch
* master

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (master)
$ git branch -M main

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 889 bytes | 444.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/Azuneer/CUB-Scripts-Administration.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (main)
$ |
```

La commande permettant l'envoi de commit locaux vers GitHub est la commande "git push"

"git commit" est donc un envoi vers le dépôt local, et "git push" est un envoi vers le dépôt distant, l'utilisation de "git commit" reste impérative avant l'envoi vers le dépôt distant.

## Partie 9 - Modification et synchronisation avec GitHub

Nous ajoutons une ligne dans notre fichier de script :

```
Write-Host "Test de la passerelle"  
Test-NetConnection 192.168.X.126
```

et créons un nouveau commit à l'aide des commandes suivantes :

```
git status  
git diff  
git add diagnostic-reseau.ps1  
git commit -m "Ajout du test de la passerelle"
```

Cette modification ne sera pas visible sur GitHub tant que nous n'avons pas utilisé la commande "git push".

## Partie 10 - Récupération et mise à jour d'un dépôt

"git init" permet d'initialiser un dépôt distant, mais en local. "git clone" permet de cloner littéralement un dépôt distant dans son état à jour.

Il est important de récupérer les dernières modifications avant de commencer à travailler sur un projet partagé car si un collaborateur ne fait pas de "git pull" avant de faire un "git push", il y aura un conflit de version.

## Partie 11 - Extension : travail avec des branches

Nous activons une branche dédiée aux diagnostics DNS :

```
git switch -c feature-diagnostic-dns  
git branch  
  
* feature-diagnostic-dns  
  main
```

Git indique la branche actuellement utilisée avec une étoile avant le nom de la branche

L'intérêt de travailler dans une branche spécifique permet de limiter les conflits et de s'organiser plus facilement entre collaborateurs sur le même projet mais pour des fonctionnalités distinctes.

## Partie 12 - Modifier le projet dans une branche

Nous ajoutons une modification de notre script et nous faisons un commit dans notre nouvelle branche fraîchement créée :

```

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (feature-diagnostic-dns)
$ git add diagnostic-reseau.ps1

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (feature-diagnostic-dns)
$ git commit -m "Ajout du test de résolution DNS"
[feature-diagnostic-dns 090973e] Ajout du test de résolution DNS
1 file changed, 3 insertions(+), 1 deletion(-)

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (feature-diagnostic-dns)
$ git log --oneline --graph --all
* 090973e (HEAD -> feature-diagnostic-dns) Ajout du test de résolution DNS
* 3595b79 (origin/main, main) Ajout du test de la passerelle
* 87b97df Ajout du diagnostic DNS
* 6c23afb Ajout du test TCP IP local
* dbd3a2a Création du script de diagnostic réseau

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (feature-diagnostic-dns)
$

```

Puis nous revenons à notre branche principale "main" avec la commande "git switch main"

La modification réalisée précédemment sera seulement présente dans la branche feature-diagnostic-dns car nous avons commit sur cette branche, et non pas sur la branche main.

## Partie 13 - Fusion d'une branche

La commande merge permet de fusionner deux branches entre elles. Nous utilisons cette commande pour fusionner la branche main et la branche feature-diagnostic-dns :

```
git merge feature-diagnostic-dns (main)
```

Puis, nous publions la branche mise à jour avec "git push".

## Partie 14 - Publier une branche et utiliser une Pull Request

Nous créons la branche "feature-informations-système", ajoutons les informations nécessaires dans notre script, puis faisons un commit et une publication de notre branche :

```

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (main)
$ git switch -c feature-informations-systeme
Switched to a new branch 'feature-informations-systeme'

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (feature-informations-systeme)
$ git add diagnostic-reseau.ps1

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (feature-informations-systeme)
$ git commit -m "Ajout des informations systèmes"
[feature-informations-systeme ef69e2e] Ajout des informations systèmes
1 file changed, 3 insertions(+), 1 deletion(-)

ADM-SRV-WAC0@ServeurWAC0 UCRT64 /c/CUB/Scripts (feature-informations-systeme)
$ git push -u origin feature-informations-systeme
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 358 bytes | 358.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature-informations-systeme' on GitHub by visiting:
remote:   https://github.com/Azuneer/CUB-Scripts-Administration/pull/new/feature-informations-systeme
remote:
To https://github.com/Azuneer/CUB-Scripts-Administration.git
 * [new branch]      feature-informations-systeme -> feature-informations-systeme
branch 'feature-informations-systeme' set up to track 'origin/feature-informations-systeme'.

```

La branche apparait bien sur notre dépôt GitHub :

The screenshot shows the GitHub 'Branches' page for a repository. At the top, there's a 'New branch' button. Below it, tabs for 'Overview', 'Yours', 'Active', 'Stale', and 'All' are visible. A search bar is present. The 'Default' section shows the 'main' branch, which is the default branch. Below that, the 'Your branches' section shows the 'feature-informations-systeme' branch, which was updated 1 minute ago. The 'Active branches' section also shows the 'feature-informations-systeme' branch, updated 1 minute ago. Each branch entry includes a table with columns for 'Branch', 'Updated', 'Check status', 'Behind/Ahead', and 'Pull request'.

Une branche uniquement locale n'est pas présente sur GitHub et sur le dépôt distant, c'est l'état de n'importe quelle branche avant un push initial. Les branches publiées sont des branches locales qui ont été poussées vers le dépôts distant.

L'intérêt d'une pull request est de pouvoir proposer l'intégration d'une nouvelle fonctionnalité dans notre branche principale. Il est préférable de faire vérifier une modification avant son intégration pour éviter tout problème de conflit de code (pour le développement de logiciels) ou autres.

The screenshot shows a GitHub Pull Request page titled 'Ajout des informations systèmes #1'. At the top, there's a 'Ready to merge' button and a 'Code' dropdown. Below that, a green button says 'Open' and a message says 'Azuneer wants to merge 1 commit into main from feature-informations-systeme'. The page shows a conversation with a comment from 'Azuneer' 1 minute ago, stating 'No description provided.' Below the comment, there's a commit titled 'Ajout des informations systèmes' with the hash 'ef69e2e'. A green box indicates 'No conflicts with base branch' and 'Merging can be performed automatically.' with a 'Merge pull request' button. On the right, there's a sidebar with sections for 'Reviewers' (No reviews), 'Assignees' (No one—assign yourself), 'Labels' (None yet), 'Projects' (None yet), 'Milestone' (No milestone), and 'Development'.

## Partie 15 - Extension : comprendre un conflit de fusion

Lors d'un conflit de fusion, Git ne peut pas déterminer automatiquement quelle modification privilégier parmi ces deux dernières, ce qui est logique car l'une n'a aucune raison valable de l'emporter sur l'autre, le choix est donc laissé à l'utilisateur lors d'un conflit.

Celui qui doit décider du contenu à conserver est le collaborateur ayant push ses dernières modifications.

Après correction du conflit, un merge permet d'enregistrer la résolution.

## Partie 16 - Synthèse

### Rôle des commandes

| Commande                       | Rôle                                                                                  |
|--------------------------------|---------------------------------------------------------------------------------------|
| <code>git init</code>          | Initialise un nouveau dépôt Git local dans le répertoire courant                      |
| <code>git status</code>        | Affiche l'état du dépôt : branche courante, fichiers suivis/modifiés/non suivis       |
| <code>git diff</code>          | Montre les différences entre le fichier local et la dernière version commitée         |
| <code>git add</code>           | Ajoute des fichiers à la zone d'indexation (pré-commit)                               |
| <code>git commit</code>        | Enregistre les fichiers indexés dans le dépôt local avec un message descriptif        |
| <code>git log --oneline</code> | Affiche l'historique des commits de façon condensée (un commit par ligne)             |
| <code>git restore</code>       | Annule les modifications non commitées d'un fichier (retour à la version enregistrée) |
| <code>git clone</code>         | Copie un dépôt distant en local (avec tout son historique)                            |
| <code>git pull</code>          | Récupère les modifications distantes et les fusionne dans la branche locale           |
| <code>git push</code>          | Envoie les commits locaux vers le dépôt distant                                       |
| <code>git branch</code>        | Liste (ou crée/supprime) les branches du dépôt                                        |
| <code>git switch</code>        | Change de branche (ou en crée une avec <code>-c</code> )                              |
| <code>git merge</code>         | Fusionne une branche dans la branche courante                                         |

### Git vs GitHub

**Git** est un outil de versionnement **local** : il gère l'historique, les branches et les commits sur la machine. **GitHub** est une plateforme **distant** qui héberge des dépôts et permet la collaboration : elle interconnecte les dépôts de millions d'utilisateurs dans le monde. Un **commit** est un instantané versionné avec un message ; une **branche** est une ligne de développement parallèle ; une **fusion** (merge) intègre une branche dans une autre ; une **pull request** propose cette intégration, en demandant une validation/relecture avant de l'accepter.

**Question : `git add` . puis `git commit` mais oubli du `git push`**

Les autres administrateurs **ne peuvent pas voir** ce commit sur GitHub. Tant que le `git push` n'est pas exécuté, les commits restent dans le dépôt local uniquement ; GitHub n'est

informé de rien.

## **Question : faut-il modifier `main` directement ou créer une branche ?**

Il faut créer une **branche dédiée**. Modifier directement la branche `main` rend la fonctionnalité non testée visible/cassée pour toute l'équipe dès le push. Une branche dédiée isole le travail, permet de tester puis de proposer une pull request pour intégrer proprement la fonctionnalité après validation.